

基于正则化判别分析的工件 X 光检测数据 特征分析与分类模型

作者：葛旭东、李悦昊、黎子豪

院系：新闻与传播学院

2026 年 6 月

摘要

X 光透射检测能在不破坏工件的前提下反映其内部的材质与结构,是工业产线上常用的质量检测手段。当一条产线同时流转多种型号的工件时,能否仅凭检测数据自动判别型号,直接关系到分拣与质量追溯的效率。据此,本文拟构建模型,对某工厂三类 A、B、C 三类工件进行分类。据题可知,每件工件在 155 个固定检测位置上给出一组连续测量值。样本共 7978 件,其中 A 类 3572 件、B 类 2643 件、C 类 1763 件,另有 D1 至 D10 共 10 件未知工件待判别。围绕特征分析、分类判别与模型评价三项要求,我们建立了一套以正则化二次判别分析为主、多个统计判别模型相互复核的可解释分类方法,并对其判别机制与稳健性作了较为完整的检验。

针对问题一,我们从整体水平与曲线波动两个角度刻画三类工件的差异。把每件工件的 155 个测量值视作一条曲线,先比较三类的全体均值与样本内标准差,再对每个检测位置做单因素方差分析,最后用主成分分析考察高维曲线的低维结构。三类的整体透射水平依次为 A 类 7.0291、B 类 6.2109、C 类 3.8578,第一主成分独占 54.17% 的方差,三类沿这一综合强度方向排开。由此看到,C 类与 A、B 两类在整体水平上已经分开,而 A 与 B 的水平十分接近,真正把二者区分开的是曲线的波动幅度,A 类的样本内标准差约为 B 类的两倍。

针对问题二,我们按假设由弱到强的顺序建立最近质心、高斯朴素贝叶斯、正则化 LDA、正则化 QDA 与 PCA-QDA 五类模型,并在分层五折交叉验证下用总准确率、平衡准确率、宏 F1 与混淆矩阵加以比较。为避免验证集信息泄漏,标准化只在每个训练折内部估计。结果与特征分析相互印证,即越能利用方差信息的模型预测越准。正则化 QDA 的总准确率、平衡准确率与宏 F1 均为 1.0000,高斯朴素贝叶斯为 0.9917,其余模型在 0.95 附近;C 类召回率在所有模型中都是 1.0000,误判几乎全部发生在 A 与 B 之间。我们以 QDA 作主分类器,其余四个模型用于一致性复核。

针对问题三,我们用同一套标准化参数把 10 件未知工件映射到特征空间,由五个模型分别判别,取众数作为结论并以模型一致数衡量可信度。判别结果为 D1、D2、D3、D10 属于 A 类,D4、D5、D6 属于 B 类,D7、D8、D9 属于 C 类,五个模型对每件工件都给出一致判别。我们进一步追问 QDA 为何能取得满分,发现 A 与 B 在样本内标准差上几乎不重叠,用阈值 $s^* = 1.105855$ 就能完全分开,其中 A 类最小值 1.139050 高于 B 类最大值 1.072661,可见判别力来自类别的方差结构而非单点的水平差异。先验、收缩参数与降维维数三组敏感性分析表明这一结论稳定。考虑到真实产线可能出现设备漂移或新型号,我们还给出基于模型一致数与 QDA 后验概率的拒判规则,以及基于 Hotelling T^2 的批次漂移监控,为部署留出风险提示的余地。

关键词: 工件分类; X 光检测; 正则化二次判别分析; 主成分分析; 交叉验证

第一章 问题重述

1.1 问题背景

X 光透射检测是工业制造中常用的无损检测手段。射线穿过工件时强度按指数规律衰减，衰减量取决于材料的密度、厚度与成分，因此透射后测得的强度携带了工件内部结构的信息 [1]。如题所示，在一个工件上沿固定路线布置 155 个检测位置，把这些位置上的测量值按位置顺序排列，即可得一条反映该工件透射特性的离散曲线。同时同一型号工件由相同工艺生产，其检测曲线形态接近，不同型号在材料或结构上的差异，会让曲线在整体水平或局部波动上表现出系统性的不同。

工厂生产线上常常同时流转多种型号的工件。若能依据检测曲线自动判别型号，就可以支撑分拣、质量追溯与产线监控，减少人工核对的成本与差错。本文研究的对象是某工厂的三类工件 A、B、C，每件工件给出 155 个检测位置上的连续测量值。初步阅读可知，三者的检测曲线既有明显可分的部分，亦有容易混淆之处，需用合适模型刻画其可分结构，由此对未知工件给出可靠判别。

1.2 问题提出

读表易知 A、B、C 三类已知工件的检测数据与 10 件未知类型工件的检测数据。每个表第一列为工件编号，其余 155 列为该工件在 155 个检测位置上的测量值。需完成的工作有三。其一，建立模型分析三类工件检测数据的特征。其二，根据所建模型判别 10 件未知工件的类别。第三对模型作出评价。

第二章 问题分析

题目为高维多分类问题。每件工件由 155 维连续向量描述，而类别只有三种。其维度远高于类别数，意味着信息可能高度冗余，需先清理噪音。因此，本文预计先探寻三类模型如何分别，并据此挑选与数据结构相符合的判别模型，最后检验其稳健性。

对于第一项工作，特征分析不宜停留在罗列统计量，而要回答两个判断模型选择的问题。其一，三类工件的差异主要体现在整体透射水平，还是体现在曲线的波动形态。若差异在水平上，刻画均值或类中心的模型就够用；若差异在波动上，就必须使用能够刻画方差与协方差的模型。其二，这种差异是集中在个别检测位置，还是表现为整条曲线的系统性平移。若是后者，155 维信息可以压缩到少数综合方向，降维既能可视化，也能为高维协方差估计减负。为回答第一个问题，我们从三类工件的全局统计画像入手，用逐位置单因素方差分析定位区分力最强的检测位置，再用主成分分析考察整体的低维结构。

对于第二项工作，分类模型的选择遵循一条主线：让模型对数据结构的假设与特征分析的发现相匹配，并保持判别过程可解释、可复核。基于这一标准，我们选取五个假设由弱到强的统计判别模型。最近质心只比较到各类中心的距离，朴素贝叶斯利用各检测位置的边缘分布及其方差，线性判别分析假设三类协方差相同因而给出线性边界，二次判别分析允许每类拥有各自的协方差因而给出二次边界，PCA 降维后的二次判别则用于检验高维结论是否稳健。把它们放在同一套交叉验证框架下比较，既能选出最契合数据的模型，又能借助模型之间的一致或分歧来度量某个判别有多可信。考虑到三类样本数量不等，评价指标在总准确率之外还需引入对各类等权的平衡准确率，以免样本最多的 A 类主导结论。

对于第三项工作，模型评价不能停在一个漂亮的准确率上。一个在交叉验证中接近满分的模型，仍要回答三个问题：它的判别力究竟来自数据的哪一种结构；这种结构在不同的参数与协议设定下是否稳定；当生产环境发生设备漂移或出现新型号时，它会不

会无声地给出错误判别。围绕这三问，我们在分类结果之外补充机制诊断与三类敏感性分析，并给出面向生产部署的拒判规则与批次漂移监控方法。

第三章 模型假设与符号说明

3.1 模型假设

为使建模过程清晰可控，我们作如下假设。

- (1) 同一类工件由相同工艺生产，其 155 维检测向量服从同一分布，类内差异由材质与测量噪声引起。
- (2) 训练样本的类别标签准确，A、B、C 三类划分无误。
- (3) 各检测位置的测量在同一标定条件下获得，训练数据与未知数据来自同一检测系统，量纲一致。
- (4) 训练集中三类样本的数量比例近似反映生产线的真实比例，因此先验概率可由样本比例估计。
- (5) 未知工件属于 A、B、C 三类之一。该假设用于本题判别；在生产推广部分另行讨论出现新类型时的拒判处理。
- (6) 训练样本可近似视为独立同分布，交叉验证的随机划分不会把强相关的同批次样本同时分入训练与验证，从而保证性能估计有效。

3.2 符号说明

符号	意义
N	训练样本总数， $N = 7978$
p	检测位置数， $p = 155$
G	类别数， $G = 3$
$\mathcal{G}$	类别集合 $\{A, B, C\}$
n_g	类别 g 的样本数
$\mathbf{x}_i$	第 i 件工件的原始检测向量， $\mathbf{x}_i \in \mathbb{R}^{155}$
x_{ij}	第 i 件工件在第 j 个检测位置的测量值
y_i	第 i 件工件的类别标签
μ_j, s_j	第 j 个检测位置在训练集上的均值与标准差
$z_{ij}, \mathbf{z}_i$	标准化后的测量值与样本向量
$\bar{x}_i, s_i, R_i, IQR_i$	单件工件检测曲线的均值、样本内标准差、极差与四分位距
$\boldsymbol{\mu}_g$	类别 g 在标准化空间的类中心
$\boldsymbol{\Sigma}, \boldsymbol{\Sigma}_g$	共享协方差矩阵与类别 g 的协方差矩阵
π_g	类别 g 的先验概率
λ, α	LDA 与 QDA 的协方差收缩参数
$\delta_g^{(\cdot)}$	各模型对类别 g 的判别函数
c_{gh}	混淆矩阵元素，真实类别 g 被判为 h 的样本数
ACC, BA	总准确率与平衡准确率

续下页

表 3-1 续页

符号	意义
F_j, η_j^2	第 j 个检测位置的方差分析 F 统计量与效应量
$\lambda_k, \mathbf{v}_k, R_k$	第 k 个主成分的特征值、载荷向量与解释率
s^*	区分 A、B 两类的样本内标准差最优阈值
T^2	用于批次漂移监控的 Hotelling 统计量

第四章 数据预处理与特征分析

4.1 数据读取与质量控制

四个工作表分别对应 A、B、C 三类与未知类型工件。读取时将第一列作为工件编号，其余 155 列作为数值特征。合并三类已知工件后得到训练矩阵 $\mathbf{X} \in \mathbb{R}^{7978 \times 155}$ 与标签向量 $\mathbf{y}$ ，其中 A、B、C 三类分别为 3572、2643、1763 件。对每一类逐元素统计缺失值，三类的缺失值数量均为 0，未知数据的列数与训练数据一致，因此无需插补。

4.2 标准化处理

不同检测位置的量值水平与波动程度不同，若直接使用原始值，量值大的位置会主导距离与协方差计算。为消除量纲影响，我们对每个检测位置做 Z-score 标准化。第 j 个位置在训练集上的均值与标准差为

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij}, \quad s_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2}, \quad (4-1)$$

标准化后的测量值为

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}. \quad (4-2)$$

在交叉验证中， μ_j 与 s_j 只能由当前训练折估计，再作用于验证折，否则验证折信息会通过标准化参数泄漏到训练过程，导致评价偏乐观。

4.3 三类工件的全局统计画像

把单件工件的 155 个检测值视为一条曲线，对第 i 件工件定义四个曲线统计量。均值 $\bar{x}_i = \frac{1}{p} \sum_j x_{ij}$ 反映整体透射水平，样本内标准差 s_i 反映这条曲线沿检测位置的波动幅度，极差 R_i 与四分位距 IQR_i 分别从最极端两点与中间一半数据的角度补充刻画离散程度。前一个统计量回答工件整体偏亮还是偏暗，后三个回答曲线是平缓还是起伏。把这两类信息分开看，正是为了判断三类工件的差异落在水平上还是波动上。三类工件的总体统计特征列于表 4-1。

表 4-1: 三类工件的全局统计特征

类别	样本数	全体均值	全体标准差	样本均值标准差	样本内标准差均值	最小值	最大值
A	3572	7.0291	1.4956	0.2040	1.4831	-0.0029	14.4913
B	2643	6.2109	0.8509	0.3236	0.7847	2.6257	10.3303
C	1763	3.8578	0.8300	0.2523	0.7868	0.6268	8.0281

这张表已经透露出两条对建模有用的线索。第一条在水平上：三类整体均值呈 A 高于 B 高于 C 的次序，A 与 B 相差 0.8182，B 与 C 相差 2.3531。C 与另外两类拉开的距离接近 A、B 之间距离的三倍，单凭整体水平就足以把 C 分出来，而 A 与 B 的水平相近，仅靠均值难以区分。第二条在波动上：A 类的样本内标准差均值为 1.4831，约是 B 类 0.7847 与 C 类 0.7868 的两倍。换句话说，A 类工件的检测曲线起伏明显比 B、C 两类剧烈。这一条很关键，它意味着即便 A 与 B 的整体水平接近，二者在曲线的波动幅度上仍存在系统差异，只要模型能够利用方差信息，就有希望把 A、B 分开。图 4-1 给出三类工件的平均检测曲线及其 ± 1 标准差范围。

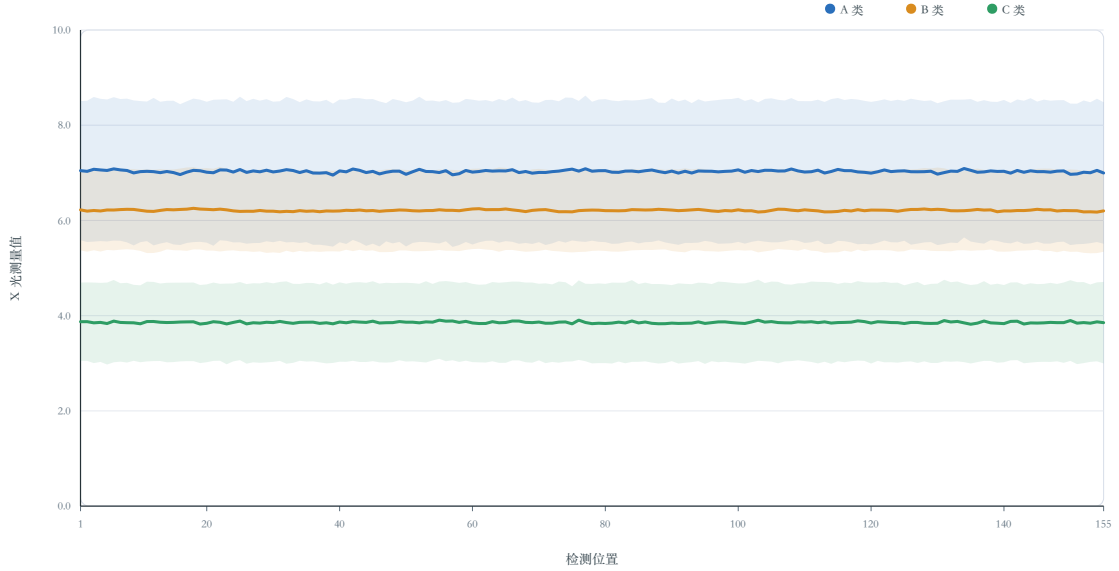


图 4-1: 三类工件平均检测曲线及 ± 1 标准差阴影

从图 4-1 可以看出，C 类曲线整体位于下方且与 A、B 两类几乎不重叠，A 类曲线位于上方且阴影最宽，B 类介于两者之间且波动较窄。这与表 4-1 的统计量一致，提示 C 类容易识别，而 A 与 B 的区分需要同时利用水平差异与波动差异。

4.4 关键检测位置的方差分析

全局画像说明三类整体可分，但还没回答差异是集中在少数检测位置，还是均匀分布在整条曲线上。这个问题决定了后续能否只靠几个关键位置做轻量化检测。单因素方差分析正适合逐位置回答它：把同一检测位置上三类的取值看成三组样本，比较组间差异与组内波动的相对大小。若组间差异远大于组内波动，说明该位置的取值主要由类别决定，区分力强。为定位对分类最有贡献的检测位置，对每个位置 j 分别做单因素方差分析。设类别 g 在位置 j 上的均值为 $\bar{x}_{gj}$ ，总体均值为 $\bar{x}_{.j}$ ，组间平方和与组内平方和为

$$SSB_j = \sum_{g \in \mathcal{G}} n_g (\bar{x}_{gj} - \bar{x}_{.j})^2, \quad SSW_j = \sum_{g \in \mathcal{G}} \sum_{i: y_i = g} (x_{ij} - \bar{x}_{gj})^2, \quad (4-3)$$

位置 j 的 F 统计量与效应量为

$$F_j = \frac{SSB_j / (G - 1)}{SSW_j / (N - G)}, \quad \eta_j^2 = \frac{SSB_j}{SSB_j + SSW_j}. \quad (4-4)$$

F_j 是组间均方与组内均方之比，越大表示该位置上类别带来的差异越压过类内噪声； $\eta_j^2 \in [0, 1]$ 则把这一差异折算成比例，表示该位置的总变异中有多大份额由类别造成，

数值越大区分力越强。两者一个看显著性，一个看效应大小，互为补充。按 F_j 排序的前 10 个检测位置列于表 4-2，全部前 30 个位置见附录 B。

表 4-2: 区分力最强的前 10 个检测位置

检测位置	A 均值	B 均值	C 均值	类均值极差	ANOVA F	η^2
134	7.0906	6.2049	3.8486	3.2419	4731.81	0.5427
138	7.0406	6.2220	3.8489	3.1917	4599.51	0.5356
88	7.0275	6.2341	3.8305	3.1971	4580.39	0.5346
108	7.0803	6.2157	3.8488	3.2315	4534.41	0.5321
75	7.0778	6.1807	3.8282	3.2495	4523.90	0.5315
125	7.0422	6.2153	3.8350	3.2071	4517.32	0.5311
78	7.0341	6.2179	3.8323	3.2017	4516.79	0.5311
62	7.0495	6.2263	3.8362	3.2133	4515.52	0.5310
139	7.0294	6.1828	3.8417	3.1877	4511.47	0.5308
89	7.0034	6.2264	3.8319	3.1716	4467.36	0.5284

结果有两点值得注意。排名前列位置的效应量都在 0.52 以上，意味着这些位置上一半以上的变异由类别造成，区分力相当强。更重要的是，这些高区分力位置在 155 个位置中分布零散（如 134、138、88、108、75 等并不相邻），且它们的类均值次序一律是 A 高于 B 高于 C，与全局画像完全一致。这说明三类差异并非由某一两个孤立位置造成，而是整条检测曲线的系统性平移：几乎每个位置都在以相近的方式把三类拉开。这一发现有两个含义，一是任意一段曲线都携带类别信息，缺少个别位置不会让分类失效；二是 155 维之间高度冗余，为后续降维埋下伏笔。图 4-2 以柱状形式展示效应量排名。

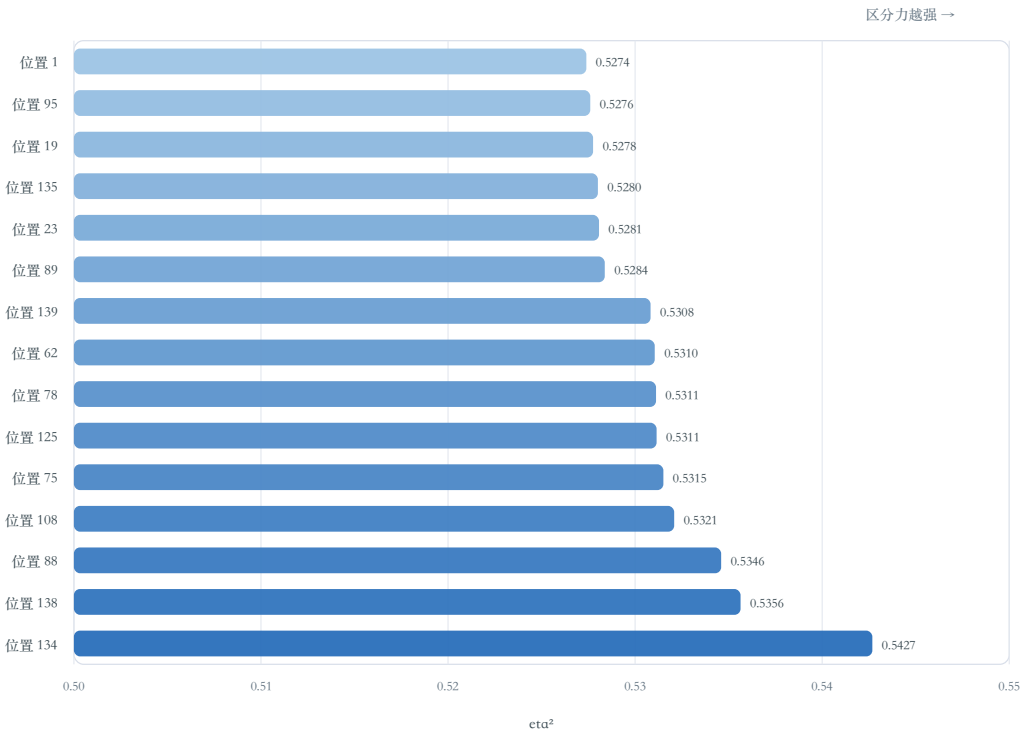


图 4-2: 关键检测位置的 η^2 排名

4.5 主成分分析与低维结构

方差分析提示 155 个位置高度冗余，主成分分析正好把这种冗余量化。它在标准化空间里寻找一组互相正交的方向，使数据沿第一个方向的方差最大、沿第二个方向在与第一个正交的前提下方差次大，依此类推。若前几个方向就吸收了绝大部分方差，就说明高维曲线实际上躺在一个低维子空间里。为判断 155 维曲线是否存在这样的低维结构，对标准化矩阵 $\mathbf{Z}$ 做主成分分析。由于 $\mathbf{Z}$ -score 标准化已使每列均值为 0 且方差为 1，标准化数据的样本协方差矩阵等于检测位置之间的相关矩阵

$$\mathbf{S} = \frac{1}{N-1} \mathbf{Z}^T \mathbf{Z}. \quad (4-5)$$

对 $\mathbf{S}$ 做特征分解 $\mathbf{S}\mathbf{v}_k = \lambda_k \mathbf{v}_k$ ，第 k 个主成分的解释率为 $R_k = \lambda_k / \sum_{\ell} \lambda_{\ell}$ 。前三个主成分的解释率分别为 0.5417、0.0050、0.0049，前十个累计解释 0.5848。第一主成分单独解释了 54.17% 的总变异，其余主成分解释率迅速下降并趋于平稳，碎石图见图 4-3。

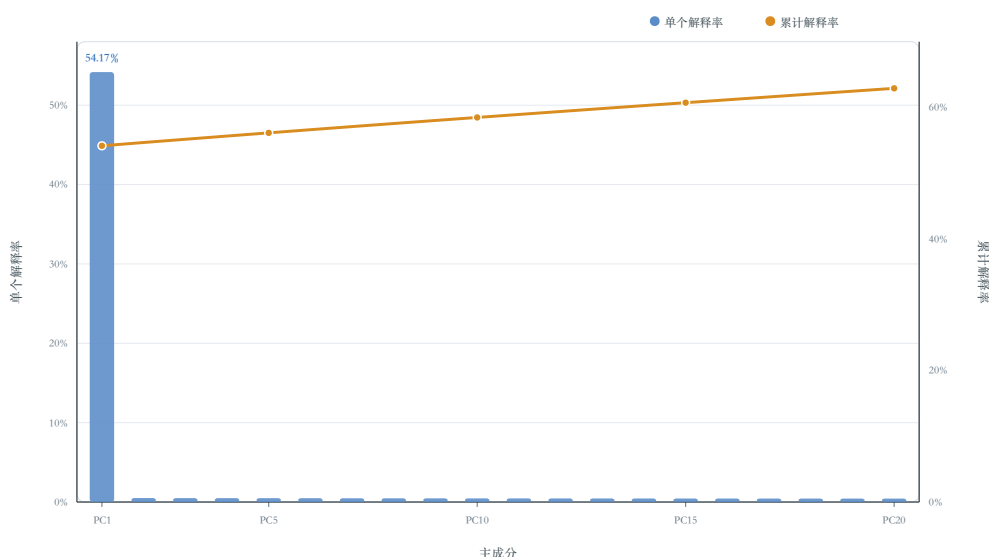


图 4-3: 主成分碎石图与累计解释率

第一主成分独占 54.17% 的方差，而第二主成分骤降到 0.50%，这种悬殊本身就在讲一个事实：155 维曲线的主要变化几乎只沿一个方向发生。结合前面各位置类均值次序一致的发现，这个方向可以理解为整条曲线的综合透射强度——它把每个位置上 A 高 B 中 C 低的同向差异汇聚成一个总指标。其余主成分解释率都在 0.5% 上下且缓慢衰减，对应的是与类别关系不大的零散噪声。

三类样本在前三个主成分上的类中心进一步印证了这一点：第一主成分上 A 类为 -7.11、B 类为 -1.13、C 类为 16.10，而第二、第三主成分上的类中心都贴近 0。也就是说，区分三类的几乎全部信息都压在第一主成分上，后续主成分基本不携带类别信息。图 4-4 给出样本在前两个主成分平面上的分布。

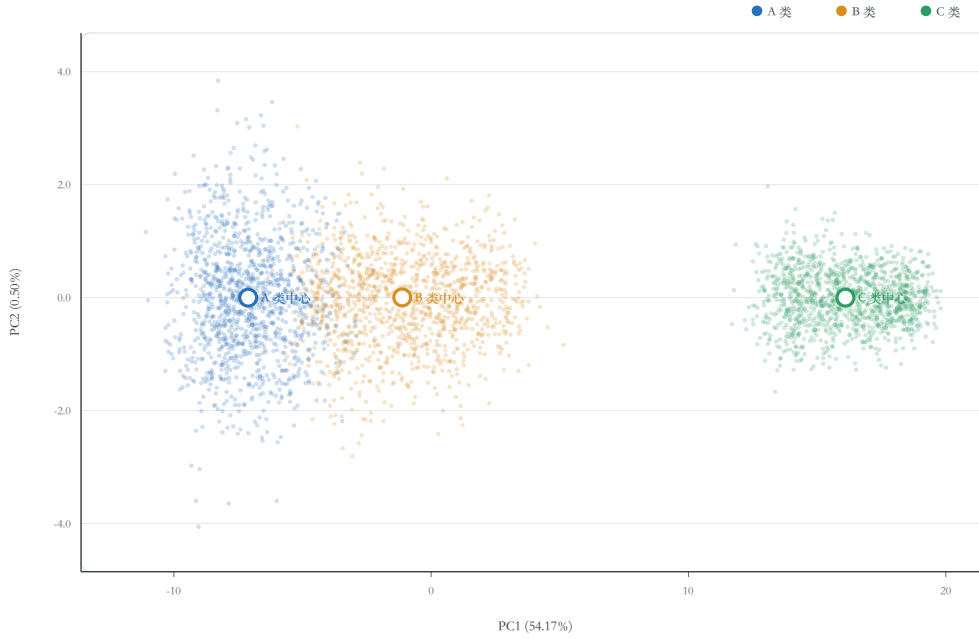


图 4-4: 前两个主成分平面上的三类工件分布与类中心

C 类点群独立位于右侧，与 A、B 两类隔开一段明显的空白；A 与 B 两类点群在第一主成分方向上部分交叠，沿第二主成分则几乎完全混在一起。这张图把前面的统计量变成了可看见的几何图景：沿“水平”这个方向，C 一眼可分，A 与 B 却你中有我。它预示 C 类在任何模型中都被正确识别，A 与 B 的区分才是分类的真正难点。同时也提醒，仅靠第一主成分这一个“水平”方向不足以分开 A 与 B，要分开它们必须回到前面发现的波动差异上，这正是后文选择二次判别分析的依据。

第五章 分类模型建立与求解

5.1 判别框架

本文的分类模型统一建立在贝叶斯判别的框架上。给定一件工件的标准化检测向量 $\mathbf{z}$ ，最优判别是把它归入后验概率最大的类别

$$\hat{y} = \arg \max_{g \in \mathcal{G}} P(y = g | \mathbf{z}). \quad (5-1)$$

由贝叶斯公式 $P(y = g | \mathbf{z}) \propto P(\mathbf{z} | y = g) \pi_g$ ，后验由类条件似然 $P(\mathbf{z} | y = g)$ 与先验 π_g 共同决定。对似然取对数不改变其大小次序，因此各模型的判别函数都可写成对数似然加对数先验的形式，差别只在于对类条件分布 $P(\mathbf{z} | y = g)$ 做了什么假设。这正是我们做模型选择的主线：用不同强度的分布假设去逼近真实的类条件分布，再看哪一种与数据最契合。

沿这条主线，我们选取五个假设由弱到强的模型。最近质心只用类中心，等价于假设各类协方差相同且为单位阵；朴素贝叶斯假设各检测位置在给定类别后相互独立，从而利用了位置级的方差；线性判别分析假设三类服从协方差相同的多元正态，给出线性边界；二次判别分析放开协方差相同这一限制，允许每类拥有各自的协方差，给出二次边界；PCA 降维后的二次判别则在低维空间重做判别，用于检验高维结论是否依赖噪声方向。这一族模型的共同优点是每一步都有清晰的统计含义，判别边界可以写成显式公式，便于向工程人员解释，也便于在出问题时逐项排查。判别分析的理论基础可追溯到 Fisher 的工作 [2]，本文采用的正则化形式来自 Friedman 提出的收缩判别分析 [3]。

5.2 最近质心模型

最近质心模型只使用类中心信息。类别 g 的中心为 $\boldsymbol{\mu}_g = \frac{1}{n_g} \sum_{i:y_i=g} \mathbf{z}_i$ ，样本到各类中心的平方距离为 $d_g^2(\mathbf{z}) = \|\mathbf{z} - \boldsymbol{\mu}_g\|_2^2$ ，判别规则取距离最小的类别

$$\hat{y}_{NC} = \arg \min_{g \in \mathcal{G}} d_g^2(\mathbf{z}). \quad (5-2)$$

几何上，最近质心把空间按到三个类中心的远近划分，两类之间的边界是连接两类中心线段的垂直平分面，因此分界完全由类中心位置决定，与各类的形状和散布无关。它假设最简单，计算最省，适合用作判断三类是否存在明显类中心分离的基线。它的弱点也正在于此：当两类均值接近而散布形状不同时，垂直平分面会从两团数据中间穿过，把本可凭波动区分的样本判错。A 与 B 恰好是这种情形，所以最近质心在 A、B 之间的表现可以作为“只用水平信息能做到多好”的参照。

5.3 高斯朴素贝叶斯

朴素贝叶斯在最近质心的基础上更进一步：它不再假设所有位置同等散布，而是允许每个位置在每一类里有各自的方差，代价是假设这些位置在给定类别后相互独立。于是类条件似然分解为各位置一维正态密度的乘积， $z_j | y = g \sim N(\mu_{gj}, \sigma_{gj}^2)$ 。把似然取对数并丢掉与类别无关的常数，再加上对数先验，得到判别函数

$$\delta_g^{NB}(\mathbf{z}) = \log \pi_g - \frac{1}{2} \sum_{j=1}^p \log \sigma_{gj}^2 - \frac{1}{2} \sum_{j=1}^p \frac{(z_j - \mu_{gj})^2}{\sigma_{gj}^2}. \quad (5-3)$$

取对数还有一个实际好处：155 个一维密度直接连乘会迅速下溢为零，化成求和后数值稳定。式中第二项 $-\frac{1}{2} \sum_j \log \sigma_{gj}^2$ 是各类的方差惩罚，第三项是按各位置方差加权的偏差平方和。正是这个加权，让朴素贝叶斯能够利用 A 类波动大、B 类波动小这一差异，而这是最近质心做不到的。先验取经验比例 $\pi_g = n_g/N$ ，代入本题为 $\pi_A = 0.4477$ 、 $\pi_B = 0.3313$ 、 $\pi_C = 0.2210$ 。

条件独立显然是一个简化：相邻检测位置之间往往相关，把它们当作独立会高估似然的“信息量”。但只要各位置的边缘分布本身已经把类别分得比较开，这种简化带来的偏差就有限，模型仍然好用，这一现象在文献中早有讨论 [4]。在本题里这个假设的代价并不均匀：后文相关性诊断显示 B 类相邻位置的相关明显高于 A、C 两类，独立假设在 B 类上偏离最大，这也预示朴素贝叶斯最可能在 B 类上失手。因此我们把朴素贝叶斯的表现读作“位置级边缘分布能把类别分到什么程度”的一个标尺。

5.4 正则化线性判别分析

线性判别分析把“各位置独立”这一过强的假设松开，转而假设每一类服从多元正态，并且三类共用同一个协方差矩阵 $\mathbf{z} | y = g \sim N(\boldsymbol{\mu}_g, \boldsymbol{\Sigma})$ 。这样位置之间的相关性被纳入模型，但三类的散布形状被强制设为相同。共享协方差由各类离差合并估计

$$\mathbf{S}_p = \frac{1}{N - G} \sum_{g \in \mathcal{G}} \sum_{i:y_i=g} (\mathbf{z}_i - \boldsymbol{\mu}_g)(\mathbf{z}_i - \boldsymbol{\mu}_g)^T. \quad (5-4)$$

把多元正态密度代入对数后验，二次项 $\mathbf{z}^T \boldsymbol{\Sigma}^{-1} \mathbf{z}$ 在三类中相同因而相消，判别函数化为关于 $\mathbf{z}$ 的线性函数，类与类之间的边界是一张超平面。

这里有一个高维带来的现实问题。协方差矩阵 $\mathbf{S}_p$ 有 $155 \times 156/2 \approx 1.2$ 万个待估元素，且前面已看到相邻位置高度相关，直接求逆会因接近奇异而极不稳定，估计出的判

别方向会被噪声放大。常用的补救是把样本协方差向其对角阵收缩，相当于在”完全相信样本相关”和”只相信各位置方差”之间取折中

$$\Sigma_{LDA} = (1 - \lambda)\mathbf{S}_p + \lambda \text{diag}(\mathbf{S}_p) + \epsilon \mathbf{I}, \quad (5-5)$$

其中 $\epsilon = 10^{-6}$ 仅用于保证正定，收缩参数 λ 越大越偏向对角、越稳健但也越接近朴素贝叶斯。 λ 在候选集合 $\{0, 0.02, 0.05, 0.08, 0.12, 0.20, 0.40, 0.60\}$ 上做网格敏感性分析（见表 7-4），准确率随 λ 增大略有上升并在 $\lambda = 0.60$ 处取得最高，故取 $\lambda = 0.60$ 。判别函数为

$$\delta_g^{LDA}(\mathbf{z}) = \mathbf{z}^T \Sigma_{LDA}^{-1} \boldsymbol{\mu}_g - \frac{1}{2} \boldsymbol{\mu}_g^T \Sigma_{LDA}^{-1} \boldsymbol{\mu}_g + \log \pi_g. \quad (5-6)$$

由于边界是线性的，LDA 能否分开 A 与 B，直接回答了”一张超平面够不够”。考虑到 A、B 的差异主要在波动而非水平，而线性边界本质上只在比较加权后的水平，可以预期 LDA 在 A、B 之间仍会力不从心，我们用它作为线性可分性的基准。

5.5 正则化二次判别分析

前面的特征分析反复指向同一件事：A 与 B 的关键差异在散布而非位置。线性判别分析受共享协方差的束缚，恰恰看不见这种散布差异，因此自然要放开这一束缚。二次判别分析假设每一类有各自的协方差 $\mathbf{z} | y = g \sim N(\boldsymbol{\mu}_g, \Sigma_g)$ ，于是不同类的二次项不再相消，判别边界变成二次曲面，能够刻画”某一类整体更分散”这类形状差异。与 LDA 同理，每类协方差按其样本量估计，仍需对角收缩以保证稳定

$$\Sigma_g^* = (1 - \alpha)\mathbf{S}_g + \alpha \text{diag}(\mathbf{S}_g) + \epsilon \mathbf{I}, \quad (5-7)$$

其中 $\mathbf{S}_g$ 为类别 g 的样本协方差。判别函数为

$$\delta_g^{QDA}(\mathbf{z}) = -\frac{1}{2}(\mathbf{z} - \boldsymbol{\mu}_g)^T (\Sigma_g^*)^{-1} (\mathbf{z} - \boldsymbol{\mu}_g) - \frac{1}{2} \log |\Sigma_g^*| + \log \pi_g. \quad (5-8)$$

为保证行列式与逆矩阵的数值稳定，对数行列式通过特征值计算

$$\log |\Sigma_g^*| = \sum_r \log \max(\lambda_{gr}, 10^{-12}), \quad (5-9)$$

其中 λ_{gr} 为 Σ_g^* 的第 r 个特征值。判别函数 (5-8) 由三部分构成，含义都很直观：第一项是样本到类中心的马氏距离，按该类自身的协方差度量”离这一类有多远”；第二项 $-\frac{1}{2} \log |\Sigma_g^*|$ 是体积惩罚，协方差越大、分布越”胖”的类越容易把远处的点也吸纳进来，这一项把这种偏好压回去；第三项是对数先验。正是第一项里每类用各自协方差度量距离，使 QDA 能据”A 的散布更大”把落在 A、B 之间的样本正确归类，这是 LDA 与最近质心都缺少的能力。收缩参数 α 在同一网格上做敏感性分析（见表 7-4），所有候选取值在交叉验证中均达到满分，说明 QDA 的满分不依赖特定收缩取值，我们取 $\alpha = 0.12$ 以兼顾协方差矩阵的数值稳定。式 (5-8) 同时利用类别均值差异与类别内部协方差结构差异，与特征分析得到的”A、B 差在波动”完全对应，因此我们以它为主分类器。

5.6 PCA-QDA 低维复核

155 维 QDA 在每类要估计一个 155×155 的协方差，参数众多，一个合理的担心是它会不会借助与类别无关的噪声方向”凑”出完美分类。PCA-QDA 用来打消这个担心：先在训练集内拟合 PCA 取前 r 个主成分构成投影矩阵 $\mathbf{V}_r$ ，把样本压到低维空间 $\mathbf{u}_i = \mathbf{V}_r^T (\mathbf{z}_i - \bar{\mathbf{z}})$ ，再在该空间重做正则化 QDA（低维收缩取 $\alpha = 0.10$ ）。低维空间丢掉了高维噪声方向，若在这里仍能得到与高维一致的判别，就说明结论靠的是真实结构而非噪声。维数 r 不取某个”最优值”，而作为稳健性旋钮在 $\{1, 2, 3, 5, 8, 10, 12, 20, 30, 50\}$ 上扫描：只要未知工件的判别在如此宽的维数范围内都不变，分类结论就不依赖人为设定的维数。降维方法的依据见 PCA 的综述文献 [5]。

5.7 一致性复核与拒判规则

记五个模型的集合为 $\mathcal{M} = \{NC, NB, LDA, QDA, PCA-QDA\}$ ，每个模型对样本给出一个预测 $h_m(\mathbf{z})$ 。由于主分类器 QDA 在交叉验证中已达到满分，我们不把多模型组合解释为独立模型带来的额外提升，而将其定位为一致性复核。综合判别取五个模型预测的众数，即得票最多的类别

$$\hat{y}_{ens} = \arg \max_{g \in \mathcal{G}} \sum_{m \in \mathcal{M}} \mathbb{1}[h_m(\mathbf{z}) = g]. \quad (5-10)$$

模型一致数 $C(\mathbf{z}) = \sum_m \mathbb{1}[h_m(\mathbf{z}) = \hat{y}_{ens}]$ 表示与最终判别一致的模型个数，用于度量分类可信度。由于 QDA 在交叉验证中取得最高的平衡准确率，当各模型出现分歧时仍以 QDA 为主判依据。考虑到生产中可能出现 A、B、C 之外的新类型或设备漂移，我们给出拒判规则：当模型一致数低于 4，或 QDA 的最大后验概率低于阈值 τ_P 时，将样本标记为待人工复检

$$C(\mathbf{z}) < 4 \quad \text{或} \quad P_{\max}^{QDA}(\mathbf{z}) < \tau_P, \quad (5-11)$$

其中 τ_P 可初始取 0.80 并用历史复检样本校准。该规则使模型在面对异常工件时不会强行给出分类。

5.8 评价指标

对类别集合 $\mathcal{G} = \{A, B, C\}$ ，混淆矩阵 $\mathbf{C} = (c_{gh})$ 的元素 $c_{gh} = \sum_i \mathbb{1}(y_i = g, \hat{y}_i = h)$ 表示真实类别为 g 却被判为 h 的样本数，矩阵行为真实类别，列为预测类别，对角元越大说明判别越准确。由混淆矩阵定义总准确率、类别召回率与类别精确率

$$ACC = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(\hat{y}_i = y_i), \quad Recall_g = \frac{c_{gg}}{\sum_h c_{gh}}, \quad Precision_g = \frac{c_{gg}}{\sum_h c_{hg}}. \quad (5-12)$$

召回率衡量某类样本被正确识别的比例，精确率衡量判为某类的样本中真正属于该类的比例。两者的调和平均为类别 F1，对各类取算术平均得到宏 F1 与宏精确率

$$F1_g = \frac{2 Precision_g Recall_g}{Precision_g + Recall_g}, \quad \text{Macro-F1} = \frac{1}{G} \sum_{g \in \mathcal{G}} F1_g. \quad (5-13)$$

由于三类样本数量不等，再引入对各类等权的平衡准确率

$$BA = \frac{1}{G} \sum_{g \in \mathcal{G}} Recall_g, \quad (5-14)$$

它避免样本量最大的 A 类主导评价，能更公平地反映样本较少的 C 类。

5.9 求解流程

对 10 件未知工件，我们先用训练集的均值与标准差把它们映射到同一标准化空间，再分别由五个模型给出预测。对每件工件，取五个预测的众数作为最终类别，并记下与最终类别一致的模型个数；当一致数低于 4，或 QDA 的最大后验概率低于阈值时，不直接输出类别而转人工复检。完整建模流程见图 5-1，判别结果在结果分析部分给出。

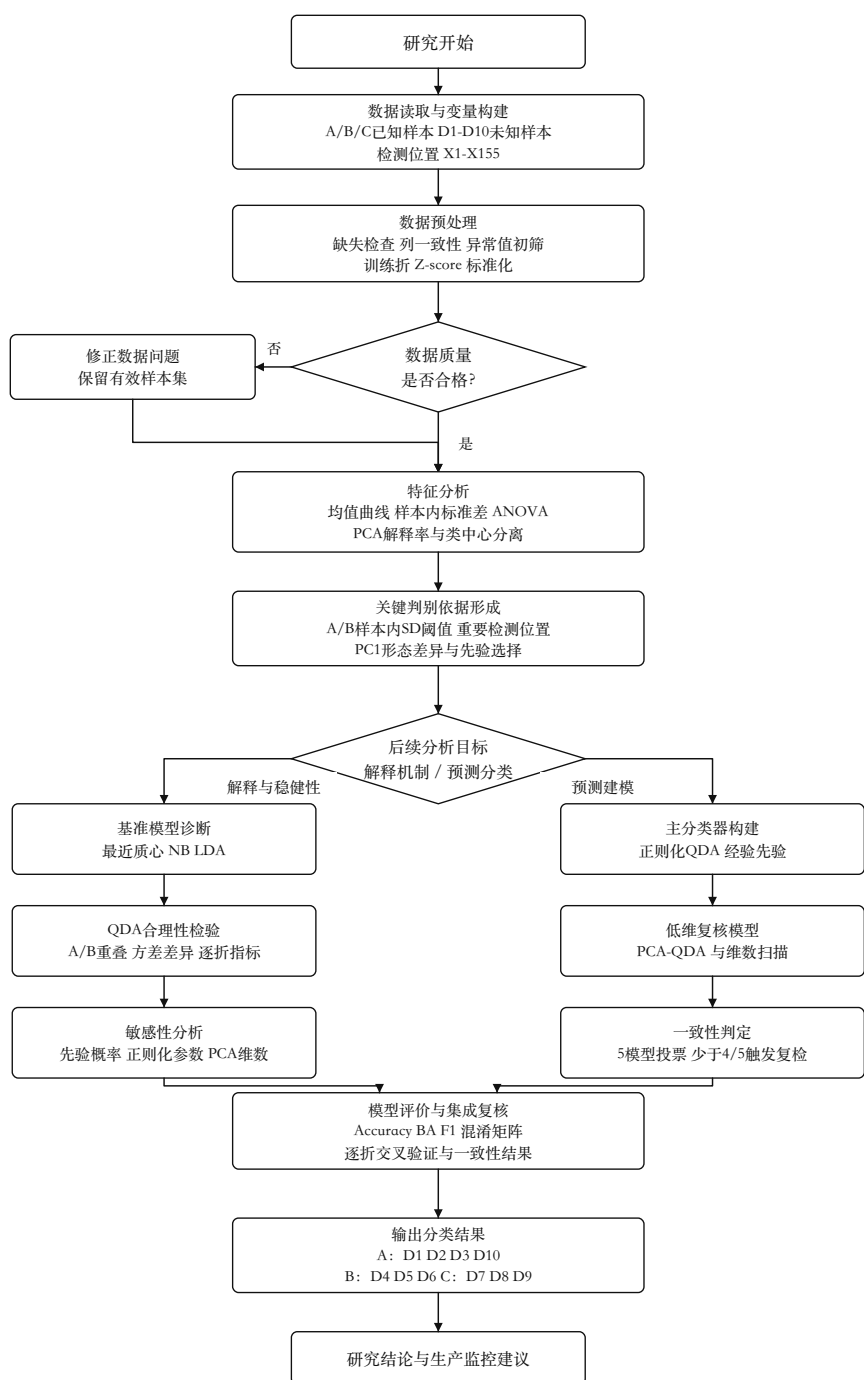


图 5-1: 本文建模流程图

第六章 结果分析

6.1 特征分析结果

特征分析回答了题目的第一项要求。三类工件的差异由两部分构成：整体透射水平的差异使 C 类与 A、B 两类明显分离，曲线波动程度的差异使 A 类区别于 B、C 两类。方差分析定位出一批效应量超过 0.52 的关键检测位置，主成分分析进一步表明类别差异主要沿第一主成分这一综合强度方向展开。这些结果共同说明，三类工件在 155 维检测空间中具有清晰的可分结构，为后续分类提供了依据。

6.2 模型性能比较

五个模型在分层五折交叉验证下的性能列于表 6-1，对应的总准确率、平衡准确率与宏 F1 见图 6-1。

表 6-1: 五个统计判别模型的交叉验证性能

模型	总准确率	宏精确率	平衡准确率	宏 F1	A 召回	B 召回	C 召回
最近质心	0.9535	0.9626	0.9564	0.9589	0.9731	0.8960	1.0000
高斯朴素贝叶斯	0.9917	0.9940	0.9917	0.9927	1.0000	0.9750	1.0000
正则化 LDA	0.9506	0.9609	0.9533	0.9563	0.9748	0.8850	1.0000
正则化 QDA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
PCA-QDA(12 维)	0.9580	0.9628	0.9639	0.9633	0.9471	0.9448	1.0000

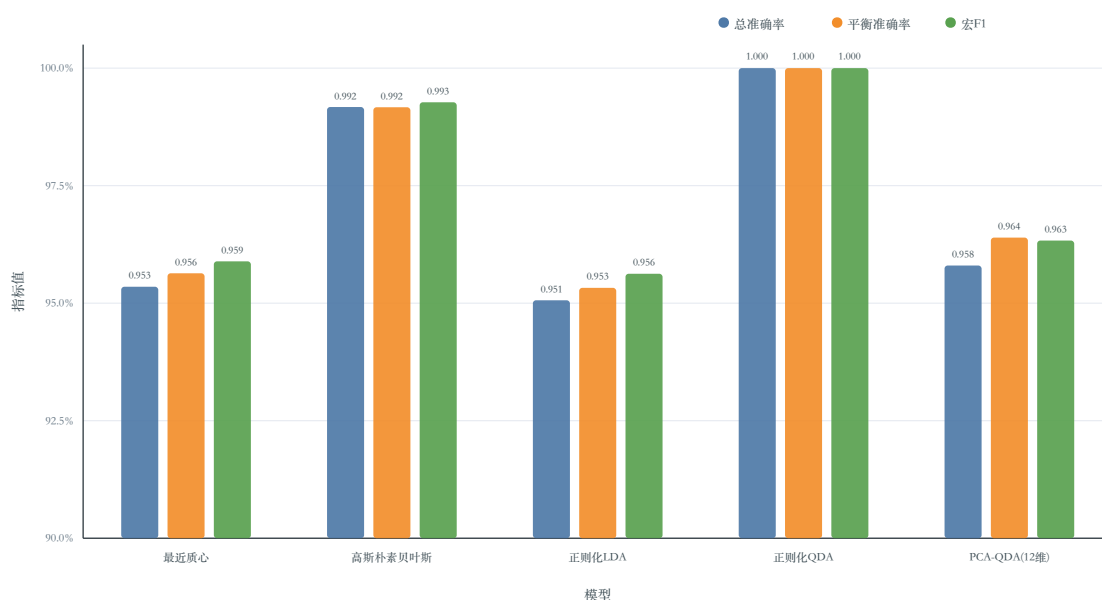


图 6-1: 五个统计判别模型的性能对比

这张表几乎是按模型假设强弱排出的一条性能曲线。最近质心与正则化 LDA 都只用到水平信息，总准确率停在 0.95 附近；朴素贝叶斯一旦用上各位置的方差，总准确率就跳到 0.9917；正则化 QDA 进一步用上每类各自的协方差，达到 1.0000。这条递进印证了前面的判断：A 与 B 的差异在方差里，谁能利用方差，谁就能把它们分开。值得一提的是朴素贝叶斯虽然假设各位置独立，却已逼近满分，这说明位置级的方差差异本身就携带了绝大部分类别信息，相关性只是锦上添花。

三类的分类别精确率、召回率与 F1 列于表 6-2，把上面的总分拆到每一类，能看得更细。C 类召回率在所有模型中都是 1.0000，与它的主成分图上独处一隅相符，任何模型都不会漏判 C。差距全部出在 A、B 之间：最近质心与 LDA 的 B 类召回率只有 0.8960 与 0.8850，意味着约一成的真 B 被误判；朴素贝叶斯把 B 类召回率提到 0.9750，QDA 则做到 1.0000。PCA-QDA(12 维) 的总准确率回落到 0.9580，是因为只保留 12 维时丢掉了部分用于区分 A、B 的方差信息，但它的价值不在准确率，而在后面要用到的稳健性复核。

表 6-2: 各模型的分类别精确率、召回率与 F1

模型	A 类			B 类			C 类		
	精确率	召回率	F1	精确率	召回率	F1	精确率	召回率	F1
最近质心	0.9267	0.9731	0.9493	0.9610	0.8960	0.9274	1.0000	1.0000	1.0000
高斯朴素贝叶斯	0.9819	1.0000	0.9908	1.0000	0.9750	0.9874	1.0000	1.0000	1.0000
正则化 LDA	0.9197	0.9748	0.9465	0.9629	0.8850	0.9223	1.0000	1.0000	1.0000
正则化 QDA	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
PCA-QDA(12 维)	0.9586	0.9471	0.9528	0.9296	0.9448	0.9371	1.0000	1.0000	1.0000

6.3 混淆结构与 A、B 方差可分机制

图 6-2 给出朴素贝叶斯、LDA 与 QDA 的混淆矩阵。三个模型有一个共同点：误判全部落在 A 与 B 之间，没有任何样本与 C 类混淆，这再次印证 C 类的独立性。误判还呈现明显的方向性，主要是真 B 被判成 A：LDA 把 304 件真 B 判为 A，朴素贝叶斯把 66 件真 B 判为 A。方向性可以这样理解，A 的散布比 B 大，按马氏距离衡量时 A 类的”势力范围”更宽，靠近边界的 B 样本更容易落进 A 的范围，只用水平或线性边界的模型尤其如此。QDA 因为分别度量到两类的马氏距离并扣除体积惩罚，纠正了这种偏向，在三类上均无误判。

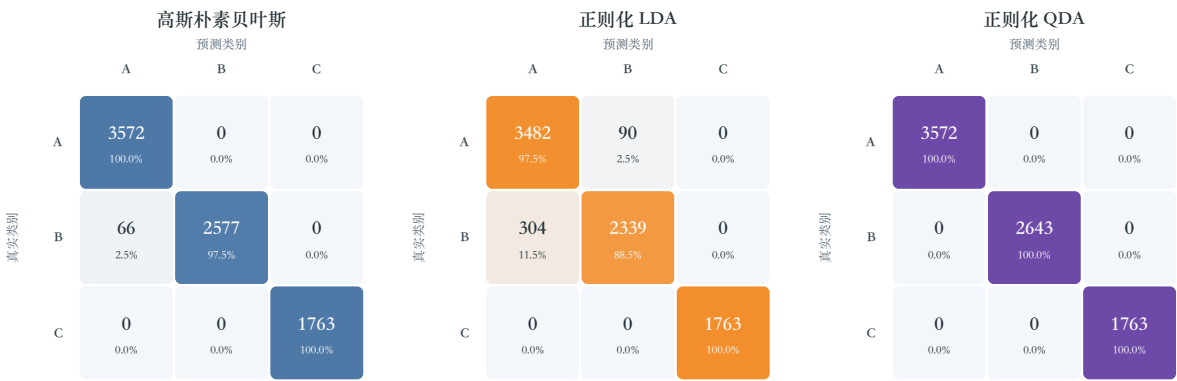


图 6-2: 朴素贝叶斯、LDA 与 QDA 的混淆矩阵

QDA 取得满分而 LDA 与最近质心不能，原因在于 A 与 B 的差异主要体现在方差而非均值。为说明这一点，对 A、B 两类的若干一维特征计算分布重叠系数与最优单阈值准确率，结果列于表 6-3。

表 6-3: A、B 两类在不同特征上的重叠与单阈值可分性

特征	直方图重叠系数	最优单阈值准确率
样本均值	0.1139	0.9426
样本内标准差	0.0000	1.0000
样本极差	0.0095	0.9955
样本四分位距	0.0066	0.9969
第一主成分得分	0.1143	0.9426
前 15 关键位置均值	0.3025	0.8484

这张表把”差异在波动里”这句话量化了。重叠系数衡量两类在某个特征上的直方图叠合程度，越接近 0 越可分；单阈值准确率则是用这一个特征、找一个最优阈值能达

到的最高 A、B 区分率。结果分成泾渭分明的两组。刻画水平的特征——样本均值与第一主成分——重叠系数约 0.11，单阈值准确率都只有约 0.9426，恰好与最近质心、LDA 的总体水平吻合，说明它们触到的正是同一个天花板：水平信息分不开 A、B。刻画波动的特征则完全不同，样本内标准差的重叠系数为 0，极差与四分位距也分别低到 0.0095 与 0.0066，单阈值准确率达到 0.9955 以上。其中样本内标准差最干净，重叠为 0、单阈值准确率为 1.0000。它的最优切割点为

$$s^* = 1.105855, \quad (6-1)$$

判别规则为 $s_i > s^*$ 判为 A 类，否则判为 B 类。训练数据中 A 类样本内标准差的最小值为 1.139050，B 类的最大值为 1.072661，两者被阈值完全隔开，因此该一维规则在训练集上实现了对 A、B 的完全分离。图 6-3 直观显示了这一分离。

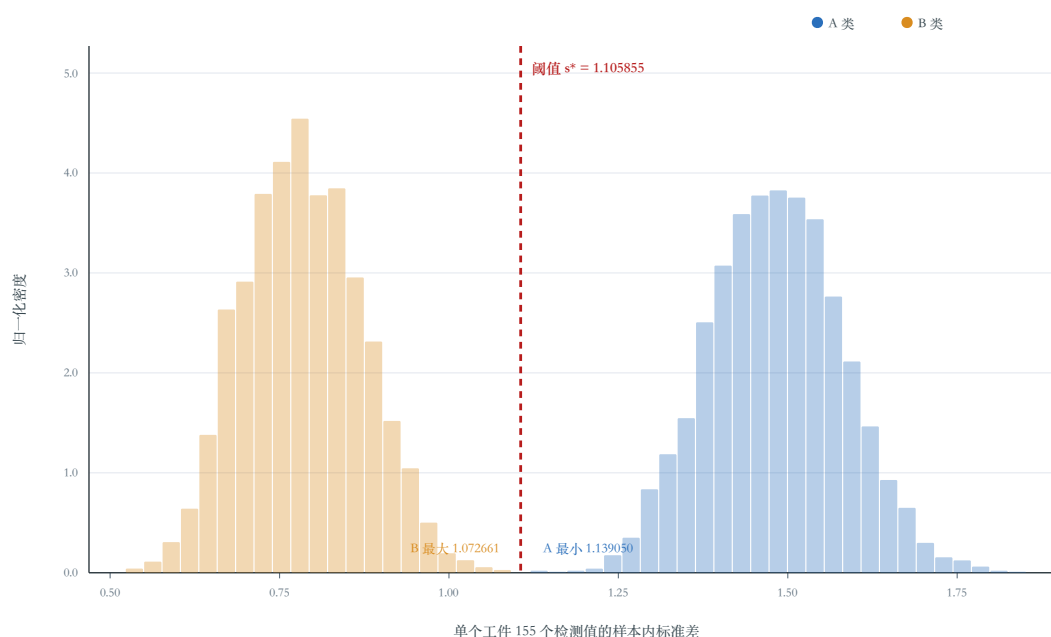


图 6-3: A、B 两类样本内标准差分布与最优阈值

这一机制解释了 QDA 满分的来源。QDA 通过类别协方差结构捕捉到 A 类波动显著大于 B 类的特征，而 LDA 假设两类协方差相同，无法利用这一差异。该结论也提示，QDA 的优势依赖检测设备的方差标定保持稳定，一旦扫描噪声或尺度发生漂移，A、B 的方差差异可能减弱，模型需要重新校准。

6.4 未知工件分类结果

10 件未知工件的判别结果列于表 6-4。五个模型对每件工件给出完全一致的判别，模型一致数均为 5。表中 QDA 最大后验概率由判别分数经 softmax 归一化得到并四舍五入为四位小数，10 件样本均为 1.0000，表明各样本到所属类别的判别分数显著高于其他两类。

表 6-4: 未知工件分类结果

工件	最终类别	一致数	最近质心	朴素贝叶斯	LDA	QDA	PCA-QDA	QDA 最大后验概率
D1	A	5/5	A	A	A	A	A	1.0000
D2	A	5/5	A	A	A	A	A	1.0000
D3	A	5/5	A	A	A	A	A	1.0000
D4	B	5/5	B	B	B	B	B	1.0000
D5	B	5/5	B	B	B	B	B	1.0000
D6	B	5/5	B	B	B	B	B	1.0000
D7	C	5/5	C	C	C	C	C	1.0000
D8	C	5/5	C	C	C	C	C	1.0000
D9	C	5/5	C	C	C	C	C	1.0000
D10	A	5/5	A	A	A	A	A	1.0000

最终判别为：D1、D2、D3、D10 属于 A 类，D4、D5、D6 属于 B 类，D7、D8、D9 属于 C 类。五个假设各异的模型给出一致结论，且 QDA 判别分数差距很大，说明在当前数据分布与交叉验证协议下，这 10 件工件均不落在类别边界附近，判别结果可信度较高。

第七章 模型检验与稳健性分析

7.1 交叉验证与各折稳定性

为避免只报告总体准确率导致评价过度乐观，表 7-1 列出五折交叉验证的各折准确率。QDA 在五折上均为 1.0000，并非某一次划分的偶然结果；其余模型各折准确率的折间极差均在约 1 个百分点以内。评价采用分层抽样保证每折的类别比例与总体一致，交叉验证用于性能估计的依据见相关方法文献 [6]。

表 7-1: 五折交叉验证的各折准确率

模型	第 1 折	第 2 折	第 3 折	第 4 折	第 5 折
最近质心	0.9549	0.9537	0.9486	0.9542	0.9561
高斯朴素贝叶斯	0.9894	0.9906	0.9906	0.9944	0.9937
正则化 LDA	0.9537	0.9499	0.9436	0.9529	0.9529
正则化 QDA	1.0000	1.0000	1.0000	1.0000	1.0000
PCA-QDA(12 维)	0.9580	0.9574	0.9536	0.9624	0.9586

7.2 先验概率敏感性

主模型采用经验先验。为检验先验选择的影响，将经验先验与等先验对比，结果列于表 7-2。先验变化对朴素贝叶斯与 LDA 仅有很小影响，对 QDA 没有任何影响，未知工件的判别结果也不改变。若训练比例不能代表生产比例，可在部署时改用等先验或生产历史比例先验。

表 7-2: 先验概率敏感性

模型	先验	总准确率	平衡准确率
朴素贝叶斯	等先验	0.9919	0.9918
朴素贝叶斯	经验先验	0.9917	0.9917
LDA	等先验	0.9521	0.9552
LDA	经验先验	0.9506	0.9533
QDA	等先验	1.0000	1.0000
QDA	经验先验	1.0000	1.0000

7.3 PCA 维数敏感性

PCA-QDA 在不同维数下的性能列于表 7-3。 $r = 12$ 并非最优维数，取该值仅作为低维稳健性检查点。提高维数能小幅改善验证准确率，但仍未达到原始 155 维 QDA 的水平。更重要的是，在全部考察的维数下，10 件未知工件的判别结果与主 QDA 完全一致，说明分类结论不依赖人为设定的降维维数。

表 7-3: PCA-QDA 在不同主成分维数下的性能

维数 r	1	2	3	5	8	10	12	20	30	50
总准确率	0.9542	0.9553	0.9565	0.9566	0.9574	0.9570	0.9580	0.9554	0.9581	0.9799
平衡准确率	0.9589	0.9601	0.9615	0.9619	0.9629	0.9628	0.9639	0.9624	0.9647	0.9831

7.4 参数稳健性

对两个含收缩参数的判别模型做网格敏感性分析，各收缩取值下的分层五折交叉验证性能列于表 7-4。LDA 的总准确率随 λ 增大的变化幅度在小数点后第三位， $\lambda = 0.60$ 处取得最高的 0.9506，故选此值。QDA 在整个网格上的总准确率与平衡准确率均为 1.0000，说明其满分不依赖特定收缩取值，取 $\alpha = 0.12$ 是出于协方差矩阵的数值稳定。需要说明的是，标准化参数与 PCA 投影都在每个训练折内部估计后再作用于验证折，从而避免验证折信息通过这些步骤泄漏到训练过程 [7]。由于三类样本数量不等，评价同时报告平衡准确率，使样本量最少的 C 类与样本量最多的 A 类获得同等权重 [8]。

表 7-4: LDA 与 QDA 的收缩参数网格敏感性

收缩参数	LDA 总准确率	LDA 平衡准确率	QDA 总准确率	QDA 平衡准确率
0.00	0.9477	0.9503	1.0000	1.0000
0.02	0.9480	0.9506	1.0000	1.0000
0.05	0.9481	0.9507	1.0000	1.0000
0.08	0.9486	0.9511	1.0000	1.0000
0.12	0.9489	0.9515	1.0000	1.0000
0.20	0.9486	0.9513	1.0000	1.0000
0.40	0.9502	0.9529	1.0000	1.0000
0.60	0.9506	0.9533	1.0000	1.0000

第八章 模型评价、改进与推广

8.1 模型优点

本文方法最突出的优点是可解释。从特征到判别再到部署，每一步都对应一个能讲清楚的统计量：方差分析指出哪些检测位置区分力强，主成分分析说明差异沿哪个方向展开，二次判别分析的马氏距离与体积惩罚解释了为什么 A、B 能分开。判别边界都有显式公式，结论可以逐项复核，这在工业现场比黑箱模型更容易被接受和维护。其次是验证完整。评价不止看一个总准确率，而是覆盖分层交叉验证、各折准确率、混淆矩阵、分类别指标与平衡准确率，并辅以先验、收缩参数与降维维数三类敏感性分析，使结论的来源与边界都摆在明处。最后是结果稳健。10 件未知工件在五个假设各异的模型下判别完全相同，主 QDA 与 r 从 1 到 50 的 PCA-QDA 也给出相同判别，说明结论既不靠某个模型的偏好，也不靠某个维数的设定。

8.2 模型局限

正则化 QDA 在交叉验证中达到满分，对此需要谨慎解释。满分说明的是，在当前这批数据与所采用的交叉验证协议下，A、B、C 三类几乎不重叠；它并不等于模型对未来任意批次都必然零错。机制分析表明这一表现主要来自 A、B 两类在方差结构上的近乎完全可分，而方差对检测设备的状态很敏感。一旦扫描噪声水平、尺度标定或采样流程发生变化，A 类波动大、B 类波动小这一差异就可能被压缩，原本干净的阈值会变模糊，模型需要重新校准。换句话说，满分是数据结构清晰的结果，而不是模型对环境变化免疫的证明。

本文的验证建立在单一数据集上，没有跨批次或跨设备的外部测试，因此上述结论的可迁移性仍需在真实产线上进一步检验。此外有两点结构性局限。其一，训练数据只包含 A、B、C 三类，若未来出现新型号工件，模型仍会把它强行归入三类之一，必须依靠拒判与人工复检兜底。其二，我们把 155 个检测位置当作无序向量处理，没有利用位置之间的空间关系；按类别计算时 A、B、C 三类相邻位置的平均相关系数分别约为 0.1189、0.6679、0.0884，B 类内部仍保留较强的相邻相关，若已知检测位置的几何排列，引入曲线平滑或函数型数据分析有望进一步压缩噪声、提升跨批次稳健性。

8.3 生产部署的拒判与漂移监控

为支撑模型在真实产线的长期运行，我们给出两项部署建议。其一是前述的拒判规则，当模型一致数低于 4 或 QDA 后验概率低于阈值时输出待复检。其二是批次漂移监控。直接在原始空间用欧氏距离监控会被量值大的位置主导，因此应先用训练集参数将新批次标准化，再在标准化空间用正则化 Hotelling 统计量

$$T_{batch}^2 = n_b(\bar{\mathbf{z}}_{batch} - \bar{\mathbf{z}}_{train})^T (\mathbf{S}_z + \eta \mathbf{I})^{-1} (\bar{\mathbf{z}}_{batch} - \bar{\mathbf{z}}_{train}), \quad (8-1)$$

其中 n_b 为批次样本数， $\bar{\mathbf{z}}_{train} = \mathbf{0}$ ， η 为小的正则化常数。控制限可由历史稳定生产期的批次 T^2 取 99% 分位数确定。当批次样本量较小时，理论近似不稳定，宜优先采用经验控制限。考虑到第一主成分解释了 54.17% 的变异，还可同步监控批次的第一主成分均值偏移 $D_{PC1} = |\bar{t}_{batch,1} - \bar{t}_{train,1}|$ 。当 T_{batch}^2 或 D_{PC1} 异常时，提示扫描尺度或工件分布可能发生漂移，应重新估计标准化参数与协方差矩阵。

8.4 模型推广

本文框架可推广到其他基于多位置传感数据的工业分类任务。当检测维度较高、且类别差异部分藏在方差结构里时，正则化 QDA 配合方差分析与主成分诊断是一种兼顾

准确率与可解释性的通用方案；它的步骤不依赖具体物理背景，换一个传感场景同样适用。

机制分析还提示了一条降低检测成本的轻量化路线。本题中区分 C 类只需要整体水平，区分 A、B 只需要曲线的波动幅度，二者都不必动用全部 155 个位置。据此可以设计一个两步判别：先用少数关键位置上的均值与一个水平阈值把 C 类分出，再对其余样本计算样本内标准差，用阈值 $s^* = 1.105855$ 区分 A 与 B。该一维标准差规则在训练集上对 A、B 的区分准确率已达 1.0000，因此这样一个只需少量位置、几乎不含矩阵运算的简化模型，有望在大幅降低采集与计算成本的同时保持接近的判别效果，适合嵌入式或在线分拣场景。在对误判代价敏感的场所，则可反向加强，把拒判阈值与各类马氏距离的分位数结合，构建一个先判别再复核的异常检测层。

第九章 结论

本文针对三类工件 X 光检测数据的特征分析与分类问题，建立了以正则化二次判别分析为主、以多种统计判别模型为复核的分类框架。特征分析表明，三类工件的整体透射水平呈 A、B、C 递减，C 类与 A、B 两类明显分离，而 A 与 B 的关键差异在于曲线波动程度。分类结果表明，正则化 QDA 在分层五折交叉验证中的各项指标均为 1.0000，其判别力来自类别方差与协方差结构，机制分析给出了 A、B 在样本内标准差上完全可分的阈值证据。对 10 件未知工件，五个模型给出一致判别，D1、D2、D3、D10 为 A 类，D4、D5、D6 为 B 类，D7、D8、D9 为 C 类。敏感性分析显示该结论对先验、正则化参数与降维维数均不敏感。本文进一步给出拒判规则与 Hotelling T^2 漂移监控方法，使模型在生产推广中具备风险提示能力。

参考文献

- [1] Mery D, Computer Vision for X-Ray Testing, Cham: Springer, 2015。
- [2] Fisher R A, The use of multiple measurements in taxonomic problems, *Annals of Eugenics*, 7(2): 179–188, 1936。
- [3] Friedman J H, Regularized discriminant analysis, *Journal of the American Statistical Association*, 84(405): 165–175, 1989。
- [4] Hand D J, Yu K, Idiot’s Bayes: not so stupid after all?, *International Statistical Review*, 69(3): 385–398, 2001。
- [5] Jolliffe I T, Cadima J, Principal component analysis: a review and recent developments, *Philosophical Transactions of the Royal Society A*, 374: 20150202, 2016。
- [6] Kohavi R, A study of cross-validation and bootstrap for accuracy estimation and model selection, *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 2: 1137–1143, 1995。
- [7] Cawley G C, Talbot N L C, On over-fitting in model selection and subsequent selection bias in performance evaluation, *Journal of Machine Learning Research*, 11: 2079–2107, 2010。
- [8] Brodersen K H, Ong C S, Stephan K E, Buhmann J M, The balanced accuracy and its posterior distribution, *Proceedings of the 20th International Conference on Pattern Recognition (ICPR)*, 3121–3124, 2010。

附录A 完整建模代码

下列代码以相对路径读取 SpecData.xlsx 的四个工作表, 实现本文全部建模过程, 包括数据读取与质量检查、标准化、特征分析、五个分类模型、分层五折交叉验证、敏感性分析与未知工件判别, 可在装有 numpy、pandas 与 openpyxl 的环境中直接运行复现本文结果。

```
1 from pathlib import Path
2 import json
3 import math
4
5 import numpy as np
6 import pandas as pd
7
8
9 BASE_DIR = Path(__file__).resolve().parent
10 DATA_FILE = BASE_DIR.parent / "2026年兰州大学数学建模竞赛赛题" / "SpecData.xlsx"
11 OUT_DIR = BASE_DIR / "outputs"
12 LABELS = ["A", "B", "C"]
13 SHEETS = {"A": "A类工件", "B": "B类工件", "C": "C类工件", "unknown": "未知类型工件"}
14 SHRINK_GRID = [0.0, 0.02, 0.05, 0.08, 0.12, 0.20, 0.40, 0.60]
15
16
17 def read_sheet(sheet):
18     df = pd.read_excel(DATA_FILE, sheet_name=sheet)
19     return df.rename(columns={df.columns[0]: "id"})
20
21
22 def load_data():
23     frames = []
24     data_meta = {}
25     for label in LABELS:
26         df = read_sheet(SHEETS[label])
27         features = list(df.columns[1:])
28         data_meta[label] = {
29             "samples": int(len(df)),
30             "features": int(len(features)),
31             "missing_values": int(df[features].isna().sum().sum()),
32         }
33         df = df.copy()
34         df["label"] = label
35         frames.append(df)
36     train = pd.concat(frames, ignore_index=True)
37     unknown = read_sheet(SHEETS["unknown"])
38     feature_cols = list(unknown.columns[1:])
39     return train, unknown, feature_cols, data_meta
40
41
42 def stratified_folds(y, k=5, seed=20260605):
43     rng = np.random.default_rng(seed)
44     folds = [[] for _ in range(k)]
45     for label in np.unique(y):
46         idx = np.where(y == label)[0]
47         rng.shuffle(idx)
48         for fold_idx, part in enumerate(np.array_split(idx, k)):
49             folds[fold_idx].extend(part.tolist())
50     return [np.array(sorted(f), dtype=int) for f in folds]
51
52
53 def standardize_train_test(X_train, X_test):
54     mu = X_train.mean(axis=0)
55     sd = X_train.std(axis=0, ddof=0)
56     sd = np.where(sd < 1e-12, 1.0, sd)
57     return (X_train - mu) / sd, (X_test - mu) / sd
58
59
60 def class_stats(X, y):
61     means, vars_, counts = {}, {}, {}
62     for label in LABELS:
63         Xi = X[y == label]
```

```

64         means[label] = Xi.mean(axis=0)
65         vars_[label] = Xi.var(axis=0, ddof=1)
66         counts[label] = Xi.shape[0]
67     return means, vars_, counts
68
69
70 def confusion_matrix(y_true, y_pred):
71     mat = np.zeros((len(LABELS), len(LABELS)), dtype=int)
72     label_to_i = {label: i for i, label in enumerate(LABELS)}
73     for yt, yp in zip(y_true, y_pred):
74         mat[label_to_i[yt], label_to_i[yp]] += 1
75     return mat
76
77
78 def metrics_from_confusion(cm):
79     precision, recall, f1 = [], [], []
80     for i in range(len(LABELS)):
81         p = cm[i, i] / cm[:, i].sum() if cm[:, i].sum() else 0.0
82         r = cm[i, i] / cm[i, :].sum() if cm[i, :].sum() else 0.0
83         f = 2 * p * r / (p + r) if p + r else 0.0
84         precision.append(float(p))
85         recall.append(float(r))
86         f1.append(float(f))
87     return {
88         "accuracy": float(np.trace(cm) / cm.sum()),
89         "macro_precision": float(np.mean(precision)),
90         "balanced_accuracy": float(np.mean(recall)),
91         "macro_f1": float(np.mean(f1)),
92         "precision": dict(zip(LABELS, precision)),
93         "recall": dict(zip(LABELS, recall)),
94         "f1": dict(zip(LABELS, f1)),
95         "confusion_matrix": cm.tolist(),
96     }
97
98
99 def softmax(scores):
100     z = scores - np.max(scores, axis=1, keepdims=True)
101     e = np.exp(np.clip(z, -700, 700))
102     return e / e.sum(axis=1, keepdims=True)
103
104
105 def predict_nearest_centroid(X_train, y_train, X_test):
106     means, _, _ = class_stats(X_train, y_train)
107     centroids = np.vstack([means[label] for label in LABELS])
108     dist2 = ((X_test[:, None, :] - centroids[None, :, :]) ** 2).sum(axis=2)
109     pred = np.array([LABELS[i] for i in np.argmin(dist2, axis=1)])
110     return pred, -dist2
111
112
113 def predict_nb(X_train, y_train, X_test, equal_prior=False):
114     means, vars_, counts = class_stats(X_train, y_train)
115     scores = []
116     for label in LABELS:
117         var = np.maximum(vars_[label], 1e-6)
118         prior = 1 / len(LABELS) if equal_prior else counts[label] / len(y_train)
119         score = math.log(prior)
120         score += -0.5 * np.sum(np.log(var))
121         score += -0.5 * ((X_test - means[label]) ** 2 / var).sum(axis=1)
122     scores.append(score)
123     scores = np.vstack(scores).T
124     pred = np.array([LABELS[i] for i in np.argmax(scores, axis=1)])
125     return pred, scores
126
127
128 def predict_lda(X_train, y_train, X_test, shrinkage=0.60, equal_prior=False):
129     means, _, counts = class_stats(X_train, y_train)
130     n, p = X_train.shape
131     pooled = np.zeros((p, p))
132     for label in LABELS:
133         centered = X_train[y_train == label] - means[label]
134         pooled += centered.T @ centered
135     pooled /= max(n - len(LABELS), 1)
136     cov = (1 - shrinkage) * pooled + shrinkage * np.diag(np.diag(pooled))

```

```

137     cov += np.eye(p) * 1e-6
138     inv_cov = np.linalg.pinv(cov)
139     scores = []
140     for label in LABELS:
141         prior = 1 / len(LABELS) if equal_prior else counts[label] / len(y_train)
142         mu = means[label]
143         score = X_test @ inv_cov @ mu
144         score += -0.5 * (mu @ inv_cov @ mu)
145         score += math.log(prior)
146         scores.append(score)
147     scores = np.vstack(scores).T
148     pred = np.array([LABELS[i] for i in np.argmax(scores, axis=1)])
149     return pred, scores
150
151
152 def predict_qda(X_train, y_train, X_test, shrinkage=0.12, equal_prior=False):
153     _, _, counts = class_stats(X_train, y_train)
154     p = X_train.shape[1]
155     scores = []
156     for label in LABELS:
157         Xi = X_train[y_train == label]
158         mu = Xi.mean(axis=0)
159         centered = Xi - mu
160         cov = centered.T @ centered / max(Xi.shape[0] - 1, 1)
161         cov = (1 - shrinkage) * cov + shrinkage * np.diag(np.diag(cov))
162         cov += np.eye(p) * 1e-6
163         inv_cov = np.linalg.pinv(cov)
164         logdet = float(np.sum(np.log(np.maximum(np.linalg.eigvalsh(cov), 1e-12))))
165         prior = 1 / len(LABELS) if equal_prior else counts[label] / len(y_train)
166         diff = X_test - mu
167         quad = np.einsum("ij,jk,ik->i", diff, inv_cov, diff)
168         scores.append(-0.5 * quad - 0.5 * logdet + math.log(prior))
169     scores = np.vstack(scores).T
170     pred = np.array([LABELS[i] for i in np.argmax(scores, axis=1)])
171     return pred, scores
172
173
174 def pca_project_train_test(X_train, X_test, r):
175     center = X_train.mean(axis=0)
176     Xc = X_train - center
177     Xt = X_test - center
178     _, _, vt = np.linalg.svd(Xc, full_matrices=False)
179     comps = vt[:r].T
180     return Xc @ comps, Xt @ comps
181
182
183 def predict_pca_qda(X_train, y_train, X_test, r=12):
184     Xtr_pc, Xte_pc = pca_project_train_test(X_train, X_test, r)
185     return predict_qda(Xtr_pc, y_train, Xte_pc, shrinkage=0.10, equal_prior=False)
186
187
188 def cross_validate(X, y, model_name, predictor):
189     folds = stratified_folds(y)
190     y_true_all, y_pred_all, fold_acc = [], [], []
191     for test_idx in folds:
192         train_idx = np.setdiff1d(np.arange(len(y)), test_idx, assume_unique=True)
193         Xtr, Xte = standardize_train_test(X[train_idx], X[test_idx])
194         pred, _ = predictor(Xtr, y[train_idx], Xte)
195         y_true_all.append(y[test_idx])
196         y_pred_all.append(pred)
197         fold_acc.append(float(np.mean(pred == y[test_idx])))
198     y_true = np.concatenate(y_true_all)
199     y_pred = np.concatenate(y_pred_all)
200     result = metrics_from_confusion(confusion_matrix(y_true, y_pred))
201     result["model"] = model_name
202     result["fold_accuracy"] = fold_acc
203     return result
204
205
206 def row_summary(X):
207     return pd.DataFrame({
208         "mean": X.mean(axis=1),
209         "sd": X.std(axis=1, ddof=1),

```



```

210         "min": X.min(axis=1),
211         "max": X.max(axis=1),
212         "median": np.median(X, axis=1),
213         "iqr": np.percentile(X, 75, axis=1) - np.percentile(X, 25, axis=1),
214         "range": X.max(axis=1) - X.min(axis=1),
215     })
216
217
218 def class_profile(train, feature_cols):
219     rows = []
220     for label in LABELS:
221         Xg = train.loc[train["label"] == label, feature_cols].to_numpy(float)
222         rs = row_summary(Xg)
223         rows.append({
224             "类别": label,
225             "样本数": len(Xg),
226             "全体均值": Xg.mean(),
227             "全体标准差": Xg.std(ddof=1),
228             "样本均值标准差": rs["mean"].std(ddof=1),
229             "样本内标准差均值": rs["sd"].mean(),
230             "最小观测值": Xg.min(),
231             "最大观测值": Xg.max(),
232         })
233     return pd.DataFrame(rows)
234
235
236 def feature_anova(X, y, feature_cols):
237     rows = []
238     overall = X.mean(axis=0)
239     total = ((X - overall) ** 2).sum(axis=0)
240     for j, col in enumerate(feature_cols):
241         groups = [X[y == label, j] for label in LABELS]
242         means = [g.mean() for g in groups]
243         between = sum(len(g) * (g.mean() - overall[j]) ** 2 for g in groups)
244         within = sum(((g - g.mean()) ** 2).sum() for g in groups)
245         F = (between / 2) / (within / (len(y) - 3))
246         eta2 = between / total[j]
247         rows.append({
248             "检测位置": col,
249             "A均值": means[0],
250             "B均值": means[1],
251             "C均值": means[2],
252             "极差(类均值)": max(means) - min(means),
253             "ANOVA_F": F,
254             "eta2": eta2,
255         })
256     return pd.DataFrame(rows).sort_values("ANOVA_F", ascending=False)
257
258
259 def pca_summary(X, y):
260     Xz = (X - X.mean(axis=0)) / np.where(X.std(axis=0, ddof=0) < 1e-12, 1, X.std(axis=0, ddof=0))
261     _, s, vt = np.linalg.svd(Xz, full_matrices=False)
262     eig = s ** 2 / (len(Xz) - 1)
263     ratio = eig / eig.sum()
264     scores = Xz @ vt[:, :3].T
265     centers = {label: scores[y == label].mean(axis=0).tolist() for label in LABELS}
266     corr = np.corrcoef(Xz, rowvar=False)
267     adjacent_corr = float(np.mean([corr[i, i + 1] for i in range(corr.shape[0] - 1)]))
268     offdiag_abs = float(np.abs(corr[np.triu_indices_from(corr, k=1)]).mean())
269     return {
270         "explained_ratio_first_10": ratio[:10].tolist(),
271         "cumulative_first_10": np.cumsum(ratio)[:10].tolist(),
272         "pc_centers": centers,
273         "adjacent_corr": adjacent_corr,
274         "offdiag_abs_corr": offdiag_abs,
275     }
276
277
278 def best_ab_threshold(values, y):
279     mask = (y == "A") | (y == "B")
280     v = values[mask]
281     yy = y[mask]

```

```

282     order = np.argsort(v)
283     vs = v[order]
284     candidates = [(vs[i] + vs[i + 1]) / 2 for i in range(len(vs) - 1)]
285     best = {"accuracy": -1}
286     for t in candidates:
287         pred = np.where(v > t, "A", "B")
288         acc = float(np.mean(pred == yy))
289         if acc > best["accuracy"]:
290             best = {"threshold": float(t), "accuracy": acc}
291     return best
292
293
294 def prior_sensitivity(X, y):
295     rows = []
296     models = {
297         "NB": lambda equal: (lambda Xtr, ytr, Xte: predict_nb(Xtr, ytr, Xte, equal_prior=equal
298         )),
299         "LDA": lambda equal: (lambda Xtr, ytr, Xte: predict_lda(Xtr, ytr, Xte, equal_prior=
300         equal)),
301         "QDA": lambda equal: (lambda Xtr, ytr, Xte: predict_qda(Xtr, ytr, Xte, equal_prior=
302         equal)),
303     }
304     for model, factory in models.items():
305         for prior_name, equal in [("等先验", True), ("经验先验", False)]:
306             r = cross_validate(X, y, model, factory(equal))
307             rows.append({
308                 "模型": model,
309                 "先验": prior_name,
310                 "总准确率": r["accuracy"],
311                 "平衡准确率": r["balanced_accuracy"],
312             })
313     return pd.DataFrame(rows)
314
315
316 def shrinkage_sensitivity(X, y, predictor, name):
317     """对收缩参数做网格敏感性分析，每个取值用同一分层五折交叉验证评价。"""
318     rows = []
319     for s in SHRINK_GRID:
320         result = cross_validate(X, y, f"{name}({s})", lambda Xtr, ytr, Xte, s=s: predictor(Xtr
321         , ytr, Xte, shrinkage=s))
322         rows.append({"模型": name, "收缩参数": s, "总准确率": result["accuracy"], "平衡准确率"
323         : result["balanced_accuracy"]})
324     return pd.DataFrame(rows)
325
326
327 def pca_r_sensitivity(X, y):
328     rows = []
329     for r in [1, 2, 3, 5, 8, 10, 12, 20, 30, 50]:
330         result = cross_validate(X, y, f"PCA-QDA({r})", lambda Xtr, ytr, Xte, r=r:
331         predict_pca_qda(Xtr, ytr, Xte, r=r))
332         rows.append({"PCA维数": r, "总准确率": result["accuracy"], "平衡准确率": result["
333         balanced_accuracy"]})
334     return pd.DataFrame(rows)
335
336
337 def predict_unknown_all(X, y, unknown_X, unknown_ids):
338     Xtr, Xu = standardize_train_test(X, unknown_X)
339     predictors = {
340         "最近质心": predict_nearest_centroid,
341         "朴素贝叶斯": predict_nb,
342         "LDA": lambda Xtr, ytr, Xte: predict_lda(Xtr, ytr, Xte, shrinkage=0.60, equal_prior=
343         False),
344         "QDA": lambda Xtr, ytr, Xte: predict_qda(Xtr, ytr, Xte, shrinkage=0.12, equal_prior=
345         False),
346         "PCA-QDA": lambda Xtr, ytr, Xte: predict_pca_qda(Xtr, ytr, Xte, r=12),
347     }
348     outputs = {}
349     qda_prob = None
350     for name, fn in predictors.items():
351         pred, scores = fn(Xtr, y, Xu)
352         outputs[name] = pred
353         if name == "QDA":
354             qda_prob = softmax(scores).max(axis=1)

```

```

346 rows = []
347 for i, item_id in enumerate(unknown_ids):
348     votes = [outputs[name][i] for name in predictors]
349     counts = {lab: votes.count(lab) for lab in LABELS}
350     top = max(counts.values())
351     winners = [lab for lab in LABELS if counts[lab] == top]
352     # 平票时以主分类器 QDA 的判别为准
353     final = outputs["QDA"][i] if len(winners) > 1 else winners[0]
354     rows.append({
355         "工件序号": item_id,
356         "预测类别": final,
357         "一致模型数": sum(v == final for v in votes),
358         "最近质心": outputs["最近质心"][i],
359         "朴素贝叶斯": outputs["朴素贝叶斯"][i],
360         "LDA": outputs["LDA"][i],
361         "QDA": outputs["QDA"][i],
362         "PCA-QDA": outputs["PCA-QDA"][i],
363         "QDA最大后验概率": qda_prob[i],
364     })
365 return pd.DataFrame(rows)
366
367
368 def md_table(df, float_digits=4):
369     headers = list(df.columns)
370     rows = []
371     for _, row in df.iterrows():
372         vals = []
373         for col in headers:
374             value = row[col]
375             if col in {"PCA维数", "一致模型数", "样本数"}:
376                 vals.append(str(int(round(float(value)))))
377             elif isinstance(value, (float, np.floating)):
378                 vals.append(f"{float(value):.{float_digits}f}")
379             else:
380                 vals.append(str(value))
381         rows.append(vals)
382     out = ["| " + " | ".join(headers) + " |", "| " + " | ".join(["---"] * len(headers)) + " |"]
383     for vals in rows:
384         out.append("| " + " | ".join(vals) + " |")
385     return "\n".join(out)
386
387
388 def write_markdown_report(paths, profile, feature_top, pca, cv_df, class_metric_df, fold_df,
389     prior_df, lda_grid_df, qda_grid_df, pca_r_df, unknown_df,
390     ab_threshold, std_stats):
391     lines = []
392     lines.append("# 工件分类数学建模最终作答\n")
393     lines.append("## 步骤 1: 数据读取与质量检查\n")
394     lines.append("训练集规模为 A 类 3572、B 类 2643、C 类 1763，共 7978 个样本；未知样本 10 个。每个工件有 155 个检测位置。三类训练数据缺失值均为 0。")
395     lines.append(md_table(profile, float_digits=4))
396     lines.append("\n\n## 步骤 2: 特征分析\n")
397     lines.append(md_table(feature_top.head(12), float_digits=4))
398     lines.append("\n\nPCA 前 3 个主成分解释率分别为 "
399         f"{pca['explained_ratio_first_10'][0]:.4f}、{pca['explained_ratio_first_10']
400         '[1]:.4f}、{pca['explained_ratio_first_10'][2]:.4f}，"
401         f"前 10 个累计解释率为 {pca['cumulative_first_10'][9]:.4f}。")
402     lines.append("\n\n## 步骤 3: 模型构建与交叉验证\n")
403     lines.append(md_table(cv_df, float_digits=6))
404     lines.append("\n\n分类别 Precision/F1: \n")
405     lines.append(md_table(class_metric_df, float_digits=6))
406     lines.append("\n\n各折准确率: \n")
407     lines.append(md_table(fold_df, float_digits=6))
408     lines.append("\n\n## 步骤 4: 敏感性分析\n")
409     lines.append("先验概率敏感性: \n")
410     lines.append(md_table(prior_df, float_digits=6))
411     lines.append("\n\nLDA 收缩参数敏感性: \n")
412     lines.append(md_table(lda_grid_df, float_digits=6))
413     lines.append("\n\nQDA 收缩参数敏感性: \n")
414     lines.append(md_table(qda_grid_df, float_digits=6))
415     lines.append("\n\nPCA 维数敏感性: \n")
416     lines.append(md_table(pca_r_df, float_digits=6))

```

```

415 lines.append(f"\n\nA/B 样本内标准差最优切割点为 {ab_threshold['threshold']:.6f}, "
416             f"A 类样本内标准差最小值为 {std_stats['A_min']:.6f}, B 类最大值为 {std_stats['B_max']:.6f}。")
417 lines.append("\n## 步骤 5: 未知工件分类\n")
418 lines.append(md_table(unknown_df, float_digits=6))
419 lines.append("\n\n最终分类: D1、D2、D3、D10 为 A; D4、D5、D6 为 B; D7、D8、D9 为 C。")
420 lines.append("\n## 输出文件\n")
421 for name, path in paths.items():
422     lines.append(f"- {name}: {path}")
423 return "\n".join(lines)
424
425
426 def main():
427     OUT_DIR.mkdir(parents=True, exist_ok=True)
428     train, unknown, feature_cols, data_meta = load_data()
429     X = train[feature_cols].to_numpy(float)
430     y = train["label"].to_numpy()
431     unknown_X = unknown[feature_cols].to_numpy(float)
432     unknown_ids = unknown["id"].astype(str).tolist()
433
434     profile = class_profile(train, feature_cols)
435     feature_rank = feature_anova(X, y, feature_cols)
436     pca = pca_summary(X, y)
437
438     model_specs = [
439         ("最近质心", predict_nearest_centroid),
440         ("高斯朴素贝叶斯", lambda Xtr, ytr, Xte: predict_nb(Xtr, ytr, Xte, equal_prior=False))
441     ],
442     ("正则化LDA", lambda Xtr, ytr, Xte: predict_lda(Xtr, ytr, Xte, shrinkage=0.60,
443         equal_prior=False)),
444     ("正则化QDA", lambda Xtr, ytr, Xte: predict_qda(Xtr, ytr, Xte, shrinkage=0.12,
445         equal_prior=False)),
446     ("PCA-QDA(12维)", lambda Xtr, ytr, Xte: predict_pca_qda(Xtr, ytr, Xte, r=12)),
447 ]
448 cv_results = [cross_validate(X, y, name, fn) for name, fn in model_specs]
449 cv_df = pd.DataFrame([
450     "模型": r["model"], "总准确率": r["accuracy"], "宏精确率": r["macro_precision"],
451     "平衡准确率": r["balanced_accuracy"], "宏F1": r["macro_f1"],
452     "A召回率": r["recall"]["A"], "B召回率": r["recall"]["B"], "C召回率": r["recall"]["C"],
453 ] for r in cv_results])
454 class_metric_df = pd.DataFrame([
455     "模型": r["model"], "A精确率": r["precision"]["A"], "A_F1": r["f1"]["A"],
456     "B精确率": r["precision"]["B"], "B_F1": r["f1"]["B"],
457     "C精确率": r["precision"]["C"], "C_F1": r["f1"]["C"],
458 ] for r in cv_results])
459 fold_df = pd.DataFrame([
460     "模型": r["model"], "Fold1": r["fold_accuracy"][0], "Fold2": r["fold_accuracy"][1],
461     "Fold3": r["fold_accuracy"][2], "Fold4": r["fold_accuracy"][3], "Fold5": r["fold_accuracy"][4],
462 ] for r in cv_results])
463 prior_df = prior_sensitivity(X, y)
464 lda_grid_df = shrinkage_sensitivity(X, y, predict_lda, "LDA")
465 qda_grid_df = shrinkage_sensitivity(X, y, predict_qda, "QDA")
466 pca_r_df = pca_r_sensitivity(X, y)
467 unknown_pred = predict_unknown_all(X, y, unknown_X, unknown_ids)
468
469 rs = row_summary(X)
470 ab_threshold = best_ab_threshold(rs["sd"].to_numpy(), y)
471 std_stats = {
472     "A_min": float(rs.loc[y == "A", "sd"].min()),
473     "B_max": float(rs.loc[y == "B", "sd"].max()),
474 }
475
476 paths = {
477     "最终作答": OUT_DIR / "final_modeling_answer.md",
478     "未知样本分类": OUT_DIR / "final_unknown_classification.csv",
479     "模型验证指标": OUT_DIR / "final_model_metrics.csv",
480     "关键检测位置": OUT_DIR / "final_feature_importance.csv",
481     "收缩参数敏感性": OUT_DIR / "final_shrinkage_grid.csv",
482     "机器可读结果": OUT_DIR / "final_results.json",
483 }
484
485 unknown_pred.to_csv(paths["未知样本分类"], index=False, encoding="utf-8-sig")

```

```

483 cv_df.to_csv(paths["模型验证指标"], index=False, encoding="utf-8-sig")
484 feature_rank.head(30).to_csv(paths["关键检测位置"], index=False, encoding="utf-8-sig")
485 pd.concat([lda_grid_df, qda_grid_df], ignore_index=True).to_csv(paths["收缩参数敏感性"],
486 index=False, encoding="utf-8-sig")
487
488 result_json = {
489     "data_meta": data_meta,
490     "pca": pca,
491     "cv_results": cv_results,
492     "lda_shrinkage_grid": lda_grid_df.to_dict(orient="records"),
493     "qda_shrinkage_grid": qda_grid_df.to_dict(orient="records"),
494     "ab_std_threshold": ab_threshold,
495     "std_stats": std_stats,
496     "unknown_predictions": unknown_pred.to_dict(orient="records"),
497 }
498 paths["机器可读结果"].write_text(json.dumps(result_json, ensure_ascii=False, indent=2),
499 encoding="utf-8")
500
501 report = write_markdown_report(
502     {k: str(v) for k, v in paths.items()},
503     profile, feature_rank, pca, cv_df, class_metric_df, fold_df,
504     prior_df, lda_grid_df, qda_grid_df, pca_r_df, unknown_pred, ab_threshold, std_stats,
505 )
506 paths["最终作答"].write_text(report, encoding="utf-8")
507
508 print("完成。主要输出：")
509 for key, path in paths.items():
510     print(f"{key}: {path}")
511 print("\n未知样本分类：")
512 print(unknown_pred[["工件序号", "预测类别", "一致模型数"]].to_string(index=False))
513
514 if __name__ == "__main__":
515     main()

```

附录B 补充结果表

表 B-1 给出方差分析中区分力最强的前 30 个检测位置，作为正文表 4-2 的补充。

表 B-1: 区分力最强的前 30 个检测位置

检测位置	A 均值	B 均值	C 均值	类均值极差	ANOVA F	η^2
134	7.0906	6.2049	3.8486	3.2419	4731.81	0.5427
138	7.0406	6.2220	3.8489	3.1917	4599.51	0.5356
88	7.0275	6.2341	3.8305	3.1971	4580.39	0.5346
108	7.0803	6.2157	3.8488	3.2315	4534.41	0.5321
75	7.0778	6.1807	3.8282	3.2495	4523.90	0.5315
125	7.0422	6.2153	3.8350	3.2071	4517.32	0.5311
78	7.0341	6.2179	3.8323	3.2017	4516.79	0.5311
62	7.0495	6.2263	3.8362	3.2133	4515.52	0.5310
139	7.0294	6.1828	3.8417	3.1877	4511.47	0.5308
89	7.0034	6.2264	3.8319	3.1716	4467.36	0.5284
23	7.0582	6.2227	3.8268	3.2314	4461.86	0.5281
135	7.0528	6.2149	3.8202	3.2326	4460.78	0.5280
19	7.0436	6.2392	3.8245	3.2191	4456.18	0.5278
95	7.0343	6.2171	3.8374	3.1969	4453.49	0.5276
1	7.0457	6.2219	3.8733	3.1725	4449.76	0.5274
109	7.0401	6.2035	3.8712	3.1690	4447.23	0.5273
107	7.0396	6.2333	3.8503	3.1893	4447.22	0.5273
10	7.0255	6.2124	3.8299	3.1956	4443.93	0.5271
145	7.0248	6.2341	3.8448	3.1800	4442.87	0.5270
61	7.0278	6.2470	3.8348	3.1930	4435.79	0.5266
7	7.0616	6.2284	3.8578	3.2037	4429.15	0.5262
63	7.0363	6.2280	3.8743	3.1621	4415.87	0.5255
112	7.0564	6.2012	3.8537	3.2027	4414.00	0.5254
5	7.0497	6.2201	3.8350	3.2147	4411.85	0.5253
73	7.0363	6.1827	3.8647	3.1716	4410.70	0.5252
122	7.0603	6.2151	3.8660	3.1944	4407.25	0.5250
22	7.0617	6.2372	3.8610	3.2006	4401.27	0.5247
47	7.0120	6.2036	3.8526	3.1594	4400.71	0.5246
42	7.0804	6.2089	3.8752	3.2051	4399.82	0.5246
128	7.0244	6.2408	3.8404	3.1841	4399.65	0.5246